

Analysis: Coin Jam

One approach here is to enumerate strings of length N and see if they are jamcoins. To iterate over potential jamcoins, we can iterate over the base-2 interpretations (odd numbers between $2^{N-1}+1$ and 2^N-1 , inclusive) and use a recursive method to convert these to different bases.

For the Small dataset, it is sufficient to use [trial division](#) to find factors of potential jamcoins. Concretely, we can look for a nontrivial divisor of an integer k by testing divisibility of every integer in increasing order from 2 to the square root of k , inclusive and stopping if we find one. Searching past the square root of the number is not needed as if k has some nontrivial divisor d , then k/d is also a non-trivial divisor and the smaller of d and k/d is at most the square root of k . A sample implementation of this in C++ looks like this:

```
long long convertBinaryToBase(int x, int base) {
    // Some languages have built-ins which make this easy.
    // For example, in Python, we can avoid recursion and
    // just return int(bin(x)[2:], base)
    if (x == 0)
        return 0;
    return base * convertBinaryToBase(x / 2, base) + (x % 2);
}

long long findFactor(long long k) {
    for (long long d = 2; d * d <= k; d++)
        if (k % d == 0)
            return d;
    return 0;
}

void printCoins(int N, int X) {
    for (long long i = (1 << N-1) + 1; X > 0; i += 2) {
        vector<long long> factors;
        for (int base = 2; base <= 10; base++) {
            long long x = convertBinaryToBase(i, base);
            long long factor = findFactor(x);
            if (!factor)
                break;
            factors.push_back(factor);
        }
        if (factors.size() < 9)
            continue;

        cout << convertBinaryToBase(i, 10);
        for (long long factor : factors)
            cout << " " << factor;
        cout << endl;
        X -= 1;
    }
}
```

Solving the large test case may present unique challenges, depending on the programming language used. We're given that $N = 32$, which means that in base 10 we'll have 32-digit

numbers, which we can't store in a 64-bit integer. These numbers are also large enough that running the trial division algorithm on a single prime would take a huge amount of time, potentially longer than the duration of this contest! While we can solve these issues by stopping the trial division early (say, after checking up to 1000) and using arbitrary precision integers, there's actually a much nicer approach!

We can make some observations about jamcoins if we look at the output to the Small dataset from our program above. Considering the first 50 jamcoins of length 16, we find that 18 of them have divisors "3 2 3 2 7 2 3 2 3" and 11 of them have divisors "3 2 5 2 7 2 3 2 11". What's the pattern in these numbers? The numbers 5, 7, 11 from the second list provide a useful hint: they're one more than their respective bases. Giving this some thought, we can notice that the second list of divisors is formed by taking the smallest prime factor of $b+1$ for each base b . This suggests that $b+1$ is always a divisor for each of these 11 jamcoins, and we can easily verify that this is true. Understanding the other common divisor list is left as an exercise for the reader.

Note that $b+1$ in base b is 11_b . A simple test exists to determine divisibility of 11 in base-10: a number is divisible by 11 if the sum of its digits in odd positions and the sum of its digits in even positions differ by a multiple of 11. This [divisibility rule](#) for 11 can be extended to a simple rule: a string of 0s and 1s which starts and ends with a 1 and has the same number of 1s at odd and even indices is divisible by $b+1$ when interpreted as a base- b number. Such a string is therefore a jamcoin, but not every jamcoin necessarily matches this condition. A stricter condition that may be easier to notice is that a string of 0s and 1s which starts and ends with a 1 is a jamcoin if all 1s are paired, i.e. it matches the regular expression $11(0|11)^*11$. For example, in any base b , $11011_b = 1001_b \times 11_b$.

The last example here also suggests a more general rule. Consider any string p of 0s and 1s with at least two characters, which starts and ends with a 1. Any string which is p repeated multiple times with any number of 0s between repetitions is a jamcoin. In the previous example we have $p = 11$ and this general rule can be expressed as the regular expression $(1[01]^*1)(0^*\backslash 1)^+$. For example, in any base b , $1110100011101011101_b = 11101_b \times 100000001000001_b$.

We can use any of these rules to mine jamcoins easily. The Python 2 code below mines jamcoins with exactly 5 pairs of 11s, which finds enough for both the Small and Large datasets.

```
def printCoins(N, X):
    # N digits, 10 1s, N-10 0s
    for i in range(N-10):
        for j in range(N-10-i):
            for k in range(N-10-i-j):
                l = N-10-i-j-k
                assert l >= 0
                template = "11{}11{}11{}11{}11"
                output = template.format("0"*i, "0"*j, "0"*k, "0"*l)
                factors = "3 2 5 2 7 2 3 2 11"
                print output, factors
                X -= 1
            if X == 0:
                return
    # If we get here, we didn't mine enough jamcoins!
    assert False
```

Analysis: Counting Sheep

The most natural approach to this problem is to simulate the process: keep track of which digits Bleatrix has seen so far, and then keep generating and checking the numbers she names until she has seen all of the digits 0-9. But are there other cases like $N = 0$ that go on forever or take an unacceptably long time?

One way to answer this question for the purposes of the problem is to just check all possible cases between 0 and 10^6 before downloading the Large dataset. With a well-written program on a reasonably fast machine, this should take only a few seconds.

More generally, it can be proven that for any $N > 0$, the sheep doesn't have to keep naming numbers for very long:

- Regarding the digit 0: The tenth number that Bleatrix names is 10 times N , and is therefore guaranteed to end in 0.
- Regarding digits 1-9: Consider the smallest power of 10 greater than N ; call it P . Once the process reaches a number at least as large as P , then the leftmost digit will take on every possible value from 1-9 as the number increases up to (or past) $9P$. No digit can be skipped, because that would require the step between successive numbers (which equals N) to be larger than P ... but we know that N is less than P because of how we chose P .

Since, by definition of P , $10N \geq P$, we will reach a number larger than P after naming at most 10 numbers, and we will reach a number greater than $9P$ after naming at most 90 numbers. So, after checking for the special case of $N = 0$, we can run our simulation without fear of running forever, running for long enough to run out of time, or overflowing even a 32-bit integer. Within the limits of the Small and Large datasets, the worst cases turn out to be 125 followed by any number of zeroes. In those cases, Bleatrix will name 72 numbers before falling asleep.

Analysis: Fractiles

This problem is more about analyzing an existing algorithm than writing a new one. Once you understand how more complex artwork depends on the original sequence, you can solve the problem with a short piece of code.

The first thing to notice is that if the original sequence is all L s, the artwork will be all L s, no matter what the value of C is. If we choose some set of tiles that all turn out to be L s for some original sequence other than all L s, then our solution is invalid, because we won't be able to tell whether the artwork was based on that original sequence or on an original sequence of all L s. This means we have to come up with a set of positions to check out such that for any original sequence besides all L s, we will see at least one G .

Small dataset

In the Small dataset, since we can check as many tiles as the length of the original sequence, we may be tempted to try to reconstruct it in full. And while this is possible (we'll get there in a moment), there is an easier alternative. The simplest solution, as it turns out, is to always output the integers 1 through K . It can be easily proved that it works with the following two-case analysis. Let us call the original sequence O , and let A_i be the artwork of complexity i for a fixed O .

1. Suppose that O starts with an L . Let us prove that each A_i starts with O . This is trivially true for $A_1 = O$. Now, if A_i starts with O , it also starts with an L , and since the transformation maps that first L into a copy of O , A_{i+1} starts with O . By induction, each A_i starts with O . Then, by checking positions 1 through K , we are checking a copy of the original sequence O , so if there are any G s in O , we will see a G .

2. Suppose instead that O starts with a G . Let us prove that each A_i starts with a G . This is trivially true for $A_1 = O$. Now, if A_i starts with a G , then A_{i+1} also starts with a G , since the transformation maps that G at the start of A_i to K G s at the start of A_{i+1} . By induction, each A_i starts with G . Then, since we are checking position 1, we will see a G .

Since we will see at least one G for any original sequence that is not all L s, and only L s for the original sequence that is all L s, we have answered the question successfully. Notice that this also proves that there is no impossible case in the Small dataset.

The proofs above hint at another possible solution for the Small dataset that gets enough information from the tiles to know the entire O . We will explain it not only because it is interesting, but also because it is a stepping stone towards a solution for the Large dataset.

We have seen that position 1 of any A_i is always equal to position 1 of O . Is there any position in A_i that is always equal to position 2 of O ? It turns out that there is, and the same is true for any position of O .

Consider position 2 of O as an example. It is position 2 in $A_1 = O$. When A_2 is produced from A_1 , the tile at position 2 of A_1 determines which tiles will appear at positions $\text{K} + 1$ through $\text{K} + \text{K}$ of A_2 . In particular, the second of those tiles, the tile at position $\text{K} + 2$ of A_2 , is the same as the tile at position 2 of A_1 . Then, it follows that position $\text{K} + 2$ of A_2 generates positions $\text{K} * (\text{K} + 2 - 1) + 1$ through $\text{K} * (\text{K} + 2)$ of A_3 , and the second of those tiles, at position $\text{K} * (\text{K} + 2 - 1) + 2$ of A_3 , is

also a copy of position 2 of O . You can follow this further to discover which position of A_C is equal to position 2, or you can write a program to do it for you. Similarly, for each position of O there is exactly one "fixed point" position in A_C that is always equal in value, and you can get those with a program by generalizing the procedure described for position 2. If you check out all of those positions, you obtain a different result for every possible O , which makes the solution valid.

Large dataset

The reasoning that we just used to find fixed points will help us solve the Large. Each position in A_i generates K positions in A_{i+1} . So, indirectly, each position in A_i also generates K^2 positions in A_{i+2} , K^3 positions in A_{i+3} , and so on. Let us say that a position in A_{i+d} is a descendant of a position p in A_i if it was generated from a position in A_{i+d-1} generated from a position in A_{i+d-2} ... generated from position p in A_i . Notice that a G in any given position of any A_i implies a G in all descendant positions. However, if there is an L in position p of A_i , a descendant position $(p - 1)*K + d$ (with $1 \leq d \leq K$) of A_{i+1} will be equal to position d of O . So, position $(p - 1)*K + d$ of A_{i+1} is an L if and only if both position p of A_i and position d of O are L s. If we take this further, we arrive at a key insight: any position of any A_i is an L if and only if a particular set of positions in O are L s.

We can find those positions by thinking about the orders in which the descendants at each level were produced. For instance, for $K=3$, position 8 of A_3 is descendant number 2 of position 3 of A_2 , which in turn is descendant number 3 of position 1 of A_1 . That means that position 8 of A_3 is L if and only if positions 2, 3 and 1 of O are all L s. So, just by looking at position 8 of A_3 , we know whether the original sequence had a G in at least one of those three positions.

Generalizing this, if we start at position p_1 of $A_1 = O$, and take its p_2 -th descendant in A_2 , and then take its p_3 -th descendant in A_3 , and so on, until taking the p_C -th descendant in A_C , we have a single position that tells us whether the original sequence has a G in positions $p_1, p_2, \dots, p_C$. And, conversely, for any position in A_C , we can find a corresponding sequence of C positions that lead to it. So, each position we check on A_C can cover up to C positions of O , and will cover exactly C positions if we make the right choice. Since we need to cover all K positions of the original sequence, that means the impossible cases are exactly those where $S*C < K$ — that is, where getting C positions out of every one of our S tile choices is still not enough. For the rest, we can assign a list of positions $[1, 2, \dots, C]$ to tile choice 1, $[C+1, C+2, 2C]$ to tile choice 2, and so on until we get to K . If the last tile choice has a list shorter than C , we can fill it up with copies of any integer between 1 and K . Now all we need to do is match each of these lists to a position in A_C , which we can do by following the descendant path (descendants of position p are always positions $(p - 1)*K + 1$ through $(p - 1)*K + K$). This simple Python code represents this idea:

```
def Solve(k, c, s):
    if c*s > k:
        return [] # returns an empty list for impossible cases
    tiles = []
    # the list for the last tile choice is filled with copies of k
    # i is the first value of the list of the current tile choice
    for i in xrange(1, k + 1, c):
        p = 1
        # j is the step in the current list [i, i+1, ..., i+C-1]
        for j in xrange(c):
            # the min fills the last tile choice's list with copies of k
            p = (p - 1) * k + min(i + j, k)
```

```
    tiles.append(p)
return tiles
```

Analysis: Revenge of the Pancakes

In the small case we have a stack of at most 10 pancakes. The total number of possible different states we can get by performing any number of flips is thus $2^{10} = 1024$. As this is a small number of states, we can solve the small test case by performing a breadth first search over this state space. For N pancakes, this can be implemented to complete within $O(N 2^N)$ time.

The large case requires a much more efficient solution. Imagine a large stack of pancakes, starting with a few happy side up, then a few blank side up, a few more happy side up, and so on. Intuitively it seems like a good idea to avoid flipping between two pancakes facing the same direction and instead try and make larger groups of pancakes facing the same direction. Define the "grouped height" of a prefix of the stack of pancakes (possibly the entire stack) to be the number of groups of contiguous pancakes facing the same direction. The goal state has a grouped height of 1. We can consider how various flips affect the grouped height of a stack of pancakes:

- If we flip a group of pancakes that has an even grouped height, then the stack's grouped height will be unchanged.
- If we flip a group of pancakes that has an odd grouped height, then the stack's grouped height will
 - increase by 1 if we flip between two pancakes facing the same direction.
 - decrease by 1 if we flip between two pancakes facing opposite directions.
 - be unchanged if we flip the entire stack.

Since the above statements categorize all possible flips, a flip can reduce the grouped height of a stack by at most 1. As the goal state has a grouped height of 1, a stack with grouped height of H requires at least $H-1$ flips to reach the goal state. Considering simple cases, for example those in the sample input, we can see that $H-1$ flips is not necessarily sufficient. A stack consisting only of blank side up pancakes has $H = 1$ but we need to flip the entire stack to make the pancakes happy side up. Given a stack where the bottom pancake is blank side up, we need to flip the entire stack at least once as the bottom pancake is only flipped when we flip the entire stack. Flipping the entire stack does not affect the grouped height of a stack so this gives us a stricter lower bound of either $H-1$ flips if the bottom pancake is happy side up or H flips if the bottom pancake is blank side up.

For a stack with $H > 1$, we can always flip the topmost group of pancakes facing the same direction to decrease the grouped height by 1. Thus the stricter lower bound is a sufficient number of flips as the following greedy strategy takes exactly that many moves: repeatedly flip the topmost group of pancakes facing in the same direction (reducing the grouped height by 1 each flip) until the grouped height is 1 and then flip the entire stack once if needed.

The problem only asks for the minimum number of flips and not which flips to make. As argued above, this is 1 less than the grouped height if the bottom pancake is happy side up and exactly the grouped height if the bottom pancake is blank side up. A sample implementation in Python is provided below, using the fact that the grouped height is one more than the number of times the string changes from + to - or vice versa.

```
def minimumFlips(pancakes):
    groupedHeight = 1 + pancakes.count('-+') + pancakes.count('+ -')
    if pancakes.endswith('-'):
        return groupedHeight
```

```
else:  
    return groupedHeight - 1
```